JPA(Jakarta Persistence API)

Dependencies required

- JPA API
- Hibernate Core
- MySQL
- Lombok

```
<dependency>
    <groupId>jakarta.persistence</groupId>
    <artifactId>jakarta.persistence-api</artifactId>
    <version>3.1.0</version> </dependency>
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.4.4.Final</version>
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.3.0</version>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.30</version>
    <scope>provided</scope>
</dependency>
```

Entity Class:

```
@Data
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private String tech;
}
```

Create Database using mysql

```
create database jpadb1;
```

persistence.xml file (src/main/resources/META-INF folder)

```
<?xml version="1.0" encoding="UTF-8"?>

<persistence xmlns="https://jakarta.ee/xml/ns/persistence" version="3.0">

    <persistence-unit name="studentPU" transaction-type="RESOURCE_LOCAL">
        <class>com.craft.Student</class>

        <properties>
            <!-- JDBC connection -->
            <property name="jakarta.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver"/>
            <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/jpadb1"/>
            <property name="jakarta.persistence.jdbc.user" value="root"/>
            <property name="jakarta.persistence.jdbc.password" value="root"/>

            <!-- Hibernate provider -->
            <property name="jakarta.persistence.provider"
value="org.hibernate.jpa.HibernatePersistenceProvider"/>
            <property name="hibernate.dialect"
value="org.hibernate.dialect.MySQLDialect"/>
            <property name="hibernate.hbm2ddl.auto" value="update"/>
            <property name="hibernate.show_sql" value="true"/>
            <property name="hibernate.format_sql" value="true"/>
        </properties>
    </persistence-unit>

</persistence>
```

Launcher Class:

```
package com.craft;

public class App {
    public static void main(String[] args) {
        Student student = new Student();
        student.setName("John Snow");
        student.setTech("Java");


        EntityManagerFactory emf =
                        Persistence.createEntityManagerFactory("studentPU");
        EntityManager em = emf.createEntityManager();

        EntityTransaction entityTransaction = em.getTransaction();

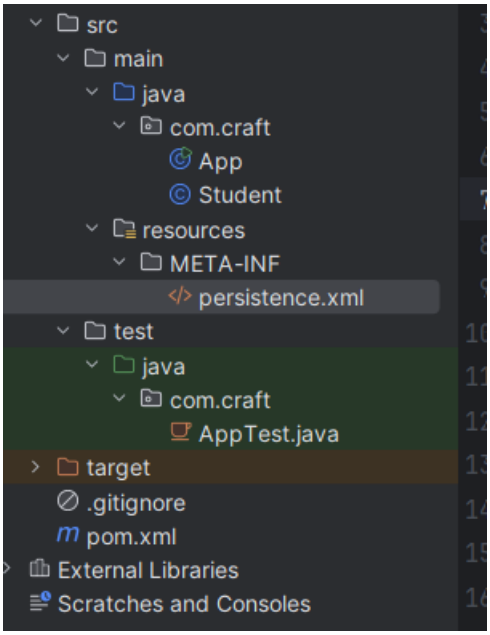
        entityTransaction.begin();

        em.persist(student);

        entityTransaction.commit();

        em.close();
    }
}
```

Project Structure So far:



CRUD

getStudentById – read

```
package com.craft;

public class App {
    public static void main(String[] args) {
        Student student = new Student();
        student.setName("John Snow");
        student.setTech("Java");


        EntityManagerFactory emf =
                  Persistence.createEntityManagerFactory("studentPU");
        EntityManager em = emf.createEntityManager();

        EntityTransaction entityTransaction = em.getTransaction();
//        entityTransaction.begin();
//        em.persist(student);

        Student fetchedStudent = em.find(Student.class, 1);

//        entityTransaction.commit();
        System.out.println("Fetched: " + fetchedStudent);
        em.close();
    }
}
```

Modular – clean code

First let's create a method to get EntityManager: getEntityManager

The factory should be class-wide as it should be created once.

```
public class App {

    public static EntityManagerFactory emf =
                  Persistence.createEntityManagerFactory("studentPU");
```

```
    public static EntityManager getEntityManager() {
        return emf.createEntityManager();

    }
```

createStudent:

```
public class App {

    public static EntityManagerFactory emf =
Persistence.createEntityManagerFactory("studentPU");

    public static EntityManager getEntityManager() {
        return emf.createEntityManager();

    }

    public static void createStudent(Student student) {
        EntityManager entityManager = getEntityManager();
        entityManager.getTransaction().begin();

        entityManager.persist(student);

        entityManager.getTransaction().commit();
    }
```

Test it:

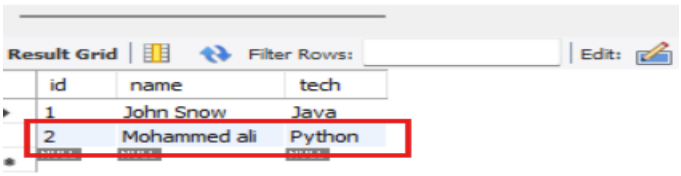
```
public static void main(String[] args) {
    Student student1 = new Student();
    student1.setName("Mohammed ali");
    student1.setTech("Python");

    createStudent(student1);

}
```

Check it in the Table:

```
create database jpadb1;
use jpadb1;
show tables;
desc student;
drop table student;
select * from student;
```



Result Grid | Filter Rows: | Edit:

id	name	tech
1	John Snow	Java
2	Mohammed ali	Python

getStudentById

```
public static Student getStudentById(int id) {

    EntityManager entityManager = getEntityManager();
    return entityManager.find(Student.class, id);
}
```

Test it:

```
public static void main(String[] args) {

    Student student1 = new Student();
    student1.setName("Mohammed ali");
    student1.setTech("Python");

//        createStudent(student1);

    System.out.println(getStudentById(2));

}
```

Output:

```
Hibernate:
    select
        s1_0.id,
        s1_0.name,
        s1_0.tech
    from
        Student s1_0
    where
        s1_0.id=?
Student(id=2, name=Mohammed ali, tech=Python)
```

Update

updateStudentName

```
public static void updateStudentName(String newName, int id) {
    EntityManager entityManager = getEntityManager();
    entityManager.getTransaction().begin();

    Student student = entityManager.find(Student.class, id);

    student.setName(newName);
    entityManager.getTransaction().commit();
}
```

Test it:

```
public static void main(String[] args) {

    Student student1 = new Student();
    student1.setName("Mohammed ali");
    student1.setTech("Python");

//        createStudent(student1);

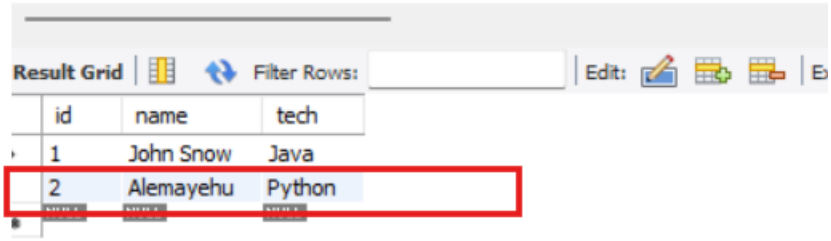
    updateStudentName("Alemayehu", 2);
    System.out.println(getStudentById(2));


}
```

Output:

```
Hibernate:
    select
        s1_0.id,
        s1_0.name,
        s1_0.tech
    from
        Student s1_0
    where
        s1_0.id=?
Hibernate:
    update
        Student
    set
        name=?,
        tech=?
    where
        id=?
Hibernate:
    select
        s1_0.id,
        s1_0.name,
        s1_0.tech
    from
        Student s1_0
    where
        s1_0.id=?
Student(id=2, name=Alemayehu, tech=Python)
```

```sql
create database jpadb1;
use jpadb1;
show tables;
desc student;
drop table student;
select * from student;
```



Result Grid | Filter Rows: | Edit:

id	name	tech
1	John Snow	Java
2	Alemayehu	Python

```java
public static void updateStudentNameAndTech(String newName, String newTech,
int id) {
    EntityManager entityManager = getEntityManager();
    entityManager.getTransaction().begin();

    Student student = entityManager.find(Student.class, id);

    student.setName(newName);
    student.setName(newTech);

    entityManager.getTransaction().commit();
}
```

deleteStudentById

```
public static void deleteStudentById(int id) {

    EntityManager entityManager = getEntityManager();
    entityManager.getTransaction().begin();
    Student student = entityManager.find(Student.class, id);
    entityManager.remove(student);
    entityManager.getTransaction().commit();
}
```

Check the **contains** method of EntityManager interface

JPQL (JPA Query Language)

1. SELECT

Let's begin with the default getStudentById method:

Default Method is:

```
public static void main(String[] args) {

        EntityManagerFactory emf =
                  Persistence.createEntityManagerFactory("studentPU");
        EntityManager em = emf.createEntityManager();

        EntityTransaction entityTransaction = em.getTransaction();

        Student fetchedStudent = em.find(Student.class, 1);

        System.out.println("Fetched: " + fetchedStudent);
        em.close();
    }
```

With JPQL

```
public static void main(String[] args) {

    EntityManager em = getEntityManager();

    TypedQuery<Student> typedQuery =
            em.createQuery("Select s From Student s where s.id = :id",
                                                    Student.class);

    typedQuery.setParameter("id", 2);

    System.out.println(typedQuery.getSingleResult());

}
```

A) Select All Students

SQL: **SELECT * FROM** student**;**

JPQL:

SELECT s FROM Student s;

```
public static void main(String[] args) {
    EntityManager em = getEntityManager();
    TypedQuery<Student> typedQuery =
            em.createQuery("From Student s", Student.class);
    List<Student> students = typedQuery.getResultList();
    System.out.println(students);
}
```

Output:

```
Hibernate:
    select
        s1_0.id,
        s1_0.name,
        s1_0.tech
    from
        Student s1_0
[Student(id=2, name=John Doe, tech=Python), Student(id=4, name=Mohammed  Ali, tech=Sport), Student(id=5, name=Firaol Mengistu, tech=soccer), Student(id=6, name=Alex Ferguson, tech=Co
```

TypedQuery, createQuery, and getResultList

1. EntityManager.createQuery() Method

Purpose

This method creates a JPQL (Java Persistence Query Language) query instance from a string-based query.

```
// For typed queries (returns TypedQuery<T>)
<T> TypedQuery<T> createQuery(String qlString, Class<T> resultClass);

// For untyped queries (returns Query)
Query createQuery(String qlString);
```

Parameters:

- **qlString**: The JPQL query string (e.g., "SELECT s FROM Student s")
- **resultClass**: The expected result type (for typed queries)

Example:

```
// Typed query
TypedQuery<Student> typedQuery =
                    em.createQuery("SELECT s FROM Student s", Student.class);

// Untyped query
Query query = em.createQuery("SELECT s FROM Student s");
```

2. TypedQuery<T> Interface

Purpose:

A type-safe extension of the Query interface that knows the type of its result.

Key Features:

- Compile-time type checking of query results
- Eliminates the need for casting
- Extends the Query interface

Important Methods (inherited from Query):

1. **getResultList**()
    - Executes the query and returns results as a **typed List**

- Returns empty list if no results found
- Throws NoResultException if used with getSingleResult()

2. **getSingleResult**()
    - Executes the query and returns a single result
    - Throws NoResultException if no results found
    - Throws NonUniqueResultException if multiple results found
3. **setParameter**() **methods**
    - For binding parameters to the query
4. **setMaxResults**()**/setFirstResult**()
    - For **pagination**

```
TypedQuery<Student> query = em.createQuery(
    "SELECT s FROM Student s WHERE s.grade = :grade", Student.class);
query.setParameter("grade", "A");

List<Student> aGradeStudents = query.getResultList();
```

3. getResultList() Method

Purpose:

Executes the query and returns the query results as a List.

Behavior:

- Returns a List containing the results (empty if no results)
- For typed queries (TypedQuery<T>), returns List<T>
- For untyped queries (Query), returns List<Object>

Important Notes:

- The returned List is not a live view of the database
- Changes to the List won't affect the database

- The List is not necessarily an ArrayList (implementation-specific)

SELECT s FROM Student s;

```
public static void main(String[] args) {
    EntityManager em = getEntityManager();
    TypedQuery<Student> typedQuery =
            em.createQuery("From Student s", Student.class);
    List<Student> students = typedQuery.getResultList();
    students.forEach(System.out::println);
}
```

This iterates through each Student object in the students list and prints it to the console.

Equivalent traditional for loop:

```
for (Student student : students) {
  System.out.println(student);
}
```

The :: Symbol (Method Reference)

What It Is:

The double colon :: is the **method reference operator** introduced in Java 8. It's used to refer to a method without invoking it.

In Your Code:

System.out::println is a method reference that points to:

- System.out - The standard output stream

- println - The method to print an object followed by a newline

Equivalent Lambda expression:

```
students.forEach(student -> System.out.println(student));
```

How It All Works Together

1. students is a List<Student> containing Student objects
2. forEach iterates through each element
3. For each Student, it calls System.out.println() with the Student as argument
4. The Student's toString() method is automatically called to get a string representation

B) Select student by tech

```
public static void main(String[] args) {

    EntityManager em = getEntityManager();
    TypedQuery<Student> typedQuery =
        em.createQuery("SELECT s From Student s WHERE s.tech = :tech",
                       Student.class);

    typedQuery.setParameter("tech", "Java");

    List<Student> javaStudents = typedQuery.getResultList();


    System.out.println(javaStudents);
}
```

C) Select Student by name and tech

```
public static void main(String[] args) {

    EntityManager em = getEntityManager();
    TypedQuery<Student> typedQuery =
```

```
                        em.createQuery("
                SELECT s
                From Student s
                WHERE s.name = :name AND s.tech = :tech", Student.class);

    typedQuery.setParameter("name", "John Doe");
    typedQuery.setParameter("tech", "Python");

    List<Student> students = typedQuery.getResultList();

    students.forEach(System.out::println);

}
```

D) Retrieve all student names

SQL: Select name from student.

JPQL: SELECT s.name
FROM Student s;

```
public static void main(String[] args) {

    EntityManager em = getEntityManager();
    TypedQuery<Student> typedQuery =
            em.createQuery("SELECT s.name From Student s", Student.class);

    List<Student> students = typedQuery.getResultList();

    System.out.println(students);

}
```

```
Hibernate:
    select
        s1_0.name
    from
        Student s1_0
[John Doe, Mohammed  Ali, Firaol Mengistu, Alex Ferguson, Yonathan]
```

Option 2:

Fetching Full Student **Objects (Then Extract Names)**

```
public static void main(String[] args) {
```

```
    EntityManager em = getEntityManager();

    TypedQuery<Student> typedQuery =
            em.createQuery("From Student s", Student.class);

    List<Student> students = typedQuery.getResultList();

    students.forEach(student -> System.out.println(student.getName()));

}
```

- **Basic Retrieval**

1. SELECT s FROM Student s
   → Fetch all students.
2. SELECT s.name FROM Student s
   → Fetch names of all students.
3. SELECT s FROM Student s WHERE s.id = 1
   → Get a student with ID = 1.
4. SELECT s FROM Student s WHERE s.name = 'John'
   → Get students named John.
5. SELECT s FROM Student s WHERE s.tech = 'Java'
   → Get all Java students.

- **LIKE, IN, BETWEEN**

6. SELECT s FROM Student s WHERE s.name LIKE 'J%'
   → Students whose names start with "J".
7. SELECT s FROM Student s WHERE s.tech IN ('Java', 'Python')
   → Students using Java or Python.
8. SELECT s FROM Student s WHERE s.id BETWEEN 5 AND 15
   → Students with IDs between 5 and 15.

- **Ordering Results**

9. SELECT s FROM Student s ORDER BY s.name ASC
   → Order students by name ascending.
10. SELECT s FROM Student s ORDER BY s.tech DESC
    → Order students by tech descending.

- **Distinct and Count**

11. SELECT DISTINCT s.tech FROM Student s
    → Unique technologies.

12. SELECT COUNT(s) FROM Student s
    → Total number of students.
13. SELECT COUNT(s) FROM Student s WHERE s.tech = 'Java'
    → Number of Java students.

- **Aggregation**

14. SELECT MAX(s.id) FROM Student s
    → Max student ID.
15. SELECT MIN(s.id) FROM Student s
    → Min student ID.
16. SELECT AVG(s.id) FROM Student s
    → Average of student IDs.
17. SELECT SUM(s.id) FROM Student s
    → Sum of student IDs (nonsensical, but legal).

- **Parameterized Queries**

18. SELECT s FROM Student s WHERE s.name = :name
    → Use named parameter :name.
19. SELECT s FROM Student s WHERE s.tech = ?1
    → Use positional parameter.

- **Update Queries**

20. UPDATE Student s SET s.tech = 'Java' WHERE s.tech = 'Python'
    → Change Python students to Java.
21. UPDATE Student s SET s.name = 'Updated' WHERE s.id = 10
    → Update name of student with ID = 10.

- **Delete Queries**

22. DELETE FROM Student s WHERE s.name = 'Test'
    → Delete students named Test.

23. DELETE FROM Student s WHERE s.tech = 'C++'
    → Delete students using C++.

- **Group By and Having**

24. SELECT s.tech, COUNT(s) FROM Student s GROUP BY s.tech
    → Count of students per tech.

25. SELECT s.tech, COUNT(s) FROM Student s GROUP BY s.tech HAVING COUNT(s) > 2
    → Technologies with more than 2 students.

- **Subqueries**

26. SELECT s FROM Student s WHERE s.id = (SELECT MAX(s2.id) FROM Student s2)
    → Student with the highest ID.

27. SELECT s FROM Student s WHERE s.tech = (SELECT s2.tech FROM Student s2 WHERE s2.id = 1)
    → Students with the same tech as student ID 1.

- **Exists / Not Exists**

28. SELECT s FROM Student s WHERE EXISTS (SELECT s2 FROM Student s2 WHERE s2.tech = s.tech AND s2.id != s.id)
    → Students who share tech with someone else.

- **Native JPQL Function Calls**

29. SELECT UPPER(s.name) FROM Student s
    → Uppercase names.

30. SELECT LENGTH(s.name) FROM Student s
    → Length of each student name.

```java
package com.craft;

import jakarta.persistence.*;
import java.util.List;

public class App {
    public static EntityManagerFactory emf =
Persistence.createEntityManagerFactory("studentPU");

    public static EntityManager getEntityManager() {
        return emf.createEntityManager();
    }

    // 1. Select all students
    public static List<Student> getAllStudents() {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery("SELECT s FROM Student s",
Student.class);
        return query.getResultList();
    }

    // 2. Select student by ID
    public static Student getStudentById(int id) {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery("SELECT s FROM Student s
WHERE s.id = :id", Student.class);
        query.setParameter("id", id);
        return query.getSingleResult();
    }

    // 3. Select students by name (exact match)
    public static List<Student> getStudentsByName(String name) {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery("SELECT s FROM Student s
WHERE s.name = :name", Student.class);
        query.setParameter("name", name);
        return query.getResultList();
    }

    // 4. Select students by name (case-insensitive)
    public static List<Student> getStudentsByNameCaseInsensitive(String name)
{
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE LOWER(s.name) = LOWER(:name)",
Student.class);
        query.setParameter("name", name);
        return query.getResultList();
    }

    // 5. Select students by technology
    public static List<Student> getStudentsByTech(String tech) {
        EntityManager em = getEntityManager();
```

```
        TypedQuery<Student> query = em.createQuery("SELECT s FROM Student s
WHERE s.tech = :tech", Student.class);
        query.setParameter("tech", tech);
        return query.getResultList();
    }

    // 6. Select students with names starting with a letter
    public static List<Student> getStudentsByNameStartingWith(String letter)
{
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.name LIKE :pattern",
Student.class);
        query.setParameter("pattern", letter + "%");
        return query.getResultList();
    }

    // 7. Select students with names containing a substring
    public static List<Student> getStudentsByNameContaining(String substring)
{
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.name LIKE :pattern",
Student.class);
        query.setParameter("pattern", "%" + substring + "%");
        return query.getResultList();
    }

    // 8. Select students with ID greater than a value
    public static List<Student> getStudentsWithIdGreaterThan(int minId) {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.id > :minId", Student.class);
        query.setParameter("minId", minId);
        return query.getResultList();
    }

    // 9. Select students with ID in a range
    public static List<Student> getStudentsWithIdBetween(int startId, int
endId) {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.id BETWEEN :startId AND :endId",
Student.class);
        query.setParameter("startId", startId);
        query.setParameter("endId", endId);
        return query.getResultList();
    }

    // 10. Select students with specific IDs (IN clause)
    public static List<Student> getStudentsByIdList(List<Integer> idList) {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.id IN :idList", Student.class);
        query.setParameter("idList", idList);
        return query.getResultList();
    }
```

```
    // 11. Select all students ordered by name (ascending)
    public static List<Student> getAllStudentsOrderByNameAsc() {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s ORDER BY s.name ASC", Student.class);
        return query.getResultList();
    }

    // 12. Select all students ordered by name (descending)
    public static List<Student> getAllStudentsOrderByNameDesc() {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s ORDER BY s.name DESC", Student.class);
        return query.getResultList();
    }

    // 13. Select first N students (pagination)
    public static List<Student> getFirstNStudents(int n) {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s ORDER BY s.id ASC", Student.class);
        query.setMaxResults(n);
        return query.getResultList();
    }

    // 14. Select students with pagination (offset)
    public static List<Student> getStudentsWithPagination(int offset, int
limit) {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s ORDER BY s.id ASC", Student.class);
        query.setFirstResult(offset);
        query.setMaxResults(limit);
        return query.getResultList();
    }

    // 15. Count all students
    public static long countAllStudents() {
        EntityManager em = getEntityManager();
        Query query = em.createQuery("SELECT COUNT(s) FROM Student s");
        return (long) query.getSingleResult();
    }

    // 16. Count students by technology
    public static List<Object[]> countStudentsByTech() {
        EntityManager em = getEntityManager();
        Query query = em.createQuery("SELECT s.tech, COUNT(s) FROM Student s
GROUP BY s.tech");
        return query.getResultList();
    }

    // 17. Find distinct technologies
    public static List<String> getDistinctTechs() {
        EntityManager em = getEntityManager();
        TypedQuery<String> query = em.createQuery("SELECT DISTINCT s.tech
FROM Student s", String.class);
```

```
        return query.getResultList();
    }

    // 18. Find average ID
    public static double getAverageId() {
        EntityManager em = getEntityManager();
        Query query = em.createQuery("SELECT AVG(s.id) FROM Student s");
        return (double) query.getSingleResult();
    }

    // 19. Find maximum ID
    public static int getMaxId() {
        EntityManager em = getEntityManager();
        Query query = em.createQuery("SELECT MAX(s.id) FROM Student s");
        return (int) query.getSingleResult();
    }

    // 20. Find students with the longest names
    public static List<Student> getStudentsWithLongestNames() {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE LENGTH(s.name) = " +
            "(SELECT MAX(LENGTH(s2.name)) FROM Student s2)", Student.class);
        return query.getResultList();
    }

    // 21. Select students with a specific technology and name pattern
    public static List<Student> getStudentsByTechAndNamePattern(String tech,
String pattern) {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.tech = :tech AND s.name LIKE
:pattern", Student.class);
        query.setParameter("tech", tech);
        query.setParameter("pattern", "%" + pattern + "%");
        return query.getResultList();
    }

    // 22. Select students with NULL technology
    public static List<Student> getStudentsWithNullTech() {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.tech IS NULL", Student.class);
        return query.getResultList();
    }

    // 23. Select students with non-NULL technology
    public static List<Student> getStudentsWithNonNullTech() {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.tech IS NOT NULL",
Student.class);
        return query.getResultList();
    }

    // 24. Update student's technology
    public static int updateStudentTech(int id, String newTech) {
```

```
        EntityManager em = getEntityManager();
        EntityTransaction tx = em.getTransaction();
        tx.begin();
        Query query = em.createQuery("UPDATE Student s SET s.tech = :newTech
WHERE s.id = :id");
        query.setParameter("newTech", newTech);
        query.setParameter("id", id);
        int updatedCount = query.executeUpdate();
        tx.commit();
        em.close();
        return updatedCount;
    }

    // 25. Update all students' technology by pattern
    public static int updateTechByPattern(String oldPattern, String newTech)
{
        EntityManager em = getEntityManager();
        EntityTransaction tx = em.getTransaction();
        tx.begin();
        Query query = em.createQuery(
            "UPDATE Student s SET s.tech = :newTech WHERE s.tech LIKE
:oldPattern");
        query.setParameter("newTech", newTech);
        query.setParameter("oldPattern", "%" + oldPattern + "%");
        int updatedCount = query.executeUpdate();
        tx.commit();
        em.close();
        return updatedCount;
    }

    // 26. Delete student by ID
    public static int deleteStudentById(int id) {
        EntityManager em = getEntityManager();
        EntityTransaction tx = em.getTransaction();
        tx.begin();
        Query query = em.createQuery("DELETE FROM Student s WHERE s.id =
:id");
        query.setParameter("id", id);
        int deletedCount = query.executeUpdate();
        tx.commit();
        em.close();
        return deletedCount;
    }

    // 27. Delete students by technology
    public static int deleteStudentsByTech(String tech) {
        EntityManager em = getEntityManager();
        EntityTransaction tx = em.getTransaction();
        tx.begin();
        Query query = em.createQuery("DELETE FROM Student s WHERE s.tech =
:tech");
        query.setParameter("tech", tech);
        int deletedCount = query.executeUpdate();
        tx.commit();
        em.close();
        return deletedCount;
    }
```

```
    // 28. Select students with the most common technology
    public static List<Student> getStudentsWithMostCommonTech() {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.tech = (" +
            "SELECT s2.tech FROM Student s2 GROUP BY s2.tech " +
            "ORDER BY COUNT(s2) DESC)", Student.class);
        query.setMaxResults(1); // Get just the most common one
        return query.getResultList();
    }

    // 29. Select students with unique technologies
    public static List<Student> getStudentsWithUniqueTechs() {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE s.tech IN (" +
            "SELECT s2.tech FROM Student s2 GROUP BY s2.tech HAVING COUNT(s2)
= 1)", Student.class);
        return query.getResultList();
    }

    // 30. Select students with name length greater than average
    public static List<Student> getStudentsWithLongerThanAverageNames() {
        EntityManager em = getEntityManager();
        TypedQuery<Student> query = em.createQuery(
            "SELECT s FROM Student s WHERE LENGTH(s.name) > " +
            "(SELECT AVG(LENGTH(s2.name)) FROM Student s2)", Student.class);
        return query.getResultList();
    }

    public static void main(String[] args) {
        // Example usage:
        List<Student> allStudents = getAllStudents();
        System.out.println("All students: " + allStudents.size());

        List<Student> javaStudents = getStudentsByTech("Java");
        System.out.println("Java students: " + javaStudents.size());

        int updated = updateStudentTech(1, "Python");
        System.out.println("Updated " + updated + " student(s)");

        // Close the EntityManagerFactory when done
        emf.close();
    }
}
```